AI-Tool reflection

I used Claude (claude.ai) throughout for problem understanding, brainstorming, and solution design, and Claude Code for code generation.

The most useful pattern I found was incremental development. Early on I asked Claude for the full solution at once: it gave me a single 600-line file covering everything. I felt lost immediately, couldn't follow the logic, and had no sense of ownership over it.

I backtracked and broke the problem into five clear iterations: taxonomy, matcher, recurrence, priority, and summary/dashboard, one module at a time, function by function, then orchestration last. That approach meant I understood every decision, could push back when something didn't fit, and left my fingerprints on the design rather than just accepting whatever was generated.

The specific moment where I overrode Claude was on recurrence detection. It suggested TF-IDF cosine similarity as a fallback, vectorising ticket descriptions and flagging similar ones from the same customer. I pushed back. At 50 rows with short descriptions, any similarity threshold would be a guess I couldn't validate, and it added a scikit-learn dependency for marginal gain. The explicit ticket references already in the description were a stronger and more transparent signal. I removed it and stuck with regex on TKT-NNN only. Claude challenged my reasoning which was useful, it forced me to articulate why simpler was actually better.

Where Claude genuinely saved time was catching edge cases early in the design conversation rather than after the code was written: the column shift bug where descriptions were landing in the wrong CSV column, subcategory casing inconsistencies, and the priority bumping logic for escalated tickets. These would have cost debugging time later. Surfacing them upfront was the real value as it can be easily missed.

AI tools are extremely good at generating code, but understanding it and staying in the driver's seat matters more: especially in a professional setting where you have to articulate design choices and defend why one solution is better than another. That judgement has to come from you, or from genuinely thinking it through with AI rather than just accepting what it produces.